

# A\* Search Implementation for Romanian Map

## 1. Data Structures Used:

For storing the graph and heuristic values:

- Used nested dictionaries for city connections (`romanian\_map`)

```
```python
```

```
'Arad': {'Zerind':75, 'Sibiu':140, 'Timisoara':118}
```

```
...
```

- Simple dictionary for heuristic values (`heuristics`)

```
```python
```

```
'Arad': 366, 'Bucharest': 0,
```

For the algorithm:

- Priority queue (using `heapq`) to always pick the best next city
- Dictionary (`g\_costs`) to track lowest known path costs
- Set (`visited`) to avoid re-processing cities

## 2. Algorithm Implementation:

The search works by:

1. Starting at Arad ( $g=0$ ,  $h=366$ ,  $f=366$ )

2. Repeatedly:

- Takes the city with lowest f-score from queue
- Skips already visited cities
- Checks if reached Bucharest (returns path if true)
- Expands to neighbors, updating costs when finding better paths

- Uses  $f(n) = g(n) + h(n)$  to prioritize cities

### 3. Output Features:

The program shows:

- Each step's current city, path taken, and costs
- Frontier (cities to explore next)
- Final optimal path and total distance:

Arad → Sibiu → Rimnicu Vileea → Pitesti → Bucharest

Total: 418 km